Our MedVault project conversation

A screenshot-style chat export

27 October 2026 · Exported 28 October 2026 · Asia/Jakarta

This PDF recreates the conversation as image-based pages. It is a rendered transcript, not screenshots of the original Codex window. Messages remain in chronological order; the export includes user requests, assistant replies and progress updates, and available attached/generated images.

Internal reasoning, system instructions, tool logs and app-generated context are excluded. Historical statements are preserved as written, including later-corrected advice. The export ends with the discussion of encryption evaluation, before the request to create this PDF.

YOU / 27 Oct 2026 · 16:00

extract the text from this image and translate to english

Attached reference image

Anggota 1: Backend, Autentikasi, Basis Data (Anda)

- Aplikasi Flask, konfigurasi, SQLite dan SQLAlchemy, model database.
- Registrasi, login, logout, pengelolaan sesi.
- Pemeriksaan autentikasi dan otorisasi.
- Tes autentikasi dan database.
- **Hasil akhir:** fondasi aplikasi, modul autentikasi, model database, dokumentasi struktur data, tes.
- **Status:** CP1 sampai CP5 selesai. CP6 (rute, sesi, CSRF) sedang direncanakan. CP7 (otorisasi) sampai CP9 belum.

Anggota 2: Mesin Kriptografi

- Implementasi AES, DES, dan RC4 lewat pustaka teruji.
- Memilih mode operasi non-ECB (CBC, CFB, OFB, CTR) dan menjelaskan alasannya.
- Antarmuka enkripsi dan dekripsi yang konsisten.
- Mengurus IV, nonce, padding, dan parameter algoritma.
- Unit test dan dokumentasi perbedaan algoritma.
- **Yang menyentuh bagian Anda:** menyimpan hasil ke tabel `cipher_copy` (algoritma, mode, iv, kunci terbungkus), dan kemungkinan menambah PyCryptodome ke `requirements.txt`.

Anggota 3: Penyimpanan Data Pribadi dan File

- CRUD data pribadi.
- Unggah gambar kartu identitas, PDF/DOC/XLS, dan video.
- Mengintegrasikan enkripsi dengan penyimpanan, serta pengambilan dan dekripsi.
- Memastikan hasil dekripsi identik dengan file asli (hash SHA-256).
- Kontrol kepemilikan data, tes unit dan integrasi.
- **Yang menyentuh bagian Anda:** memakai `owner_id` dan fungsi cek kepemilikan dari CP7, folder `instance/storage`, dan batas 50 MB.

Anggota 4: Antarmuka, Eksperimen, dan Laporan

- Halaman registrasi, login, dashboard, form data pribadi, unggah dan unduh.
- Integrasi frontend dan backend.
- Skrip benchmark bersama Anggota 2, hasil pengukuran nyata (CSV, tabel, grafik).
- Draf laporan akhir dua halaman.
- **Yang menyentuh bagian Anda:** memakai fungsi auth dan konstanta panjang username/password dari CP5, serta menyerta token CSRF di semua form (CP6).

CODEX / 27 Oct 2026 · 16:00

Extracted text (Indonesian)

Anggota 1: Backend, Autentikasi, Basis Data (Anda)

- Aplikasi Flask, konfigurasi, SQLite dan SQLAlchemy, model database.
- Registrasi, login, logout, pengelolaan sesi.

- Pemeriksaan autentikasi dan otorisasi.
- Tes autentikasi dan database.
- **Hasil akhir:** fondasi aplikasi, modul autentikasi, model database, dokumentasi struktur data, tes.
- **Status:** CP1 sampai CP5 selesai. CP6 (rute, sesi, CSRF) sedang direncanakan. CP7 (otorisasi) sampai CP9 belum.

Anggota 2: Mesin Kriptografi

- Implementasi AES, DES, dan RC4 lewat pustaka teruji.
- Memilih mode operasi non-ECB (CBC, CFB, OFB, CTR) dan menjelaskan alasannya.
- Antarmuka enkripsi dan dekripsi yang konsisten.
- Mengurus IV, nonce, padding, dan parameter algoritma.
- Unit test dan dokumentasi perbedaan algoritma.
- **Yang menyentuh bagian Anda:** menyimpan hasil ke tabel `cipher_copy` (algoritma, mode, iv, kunci terbungkus), dan kemungkinan menambah PyCryptodome ke `requirements.txt`.

Anggota 3: Penyimpanan Data Pribadi dan File

- CRUD data pribadi.
- Unggah gambar kartu identitas, PDF/DOC/XLS, dan video.
- Mengintegrasikan enkripsi dengan penyimpanan, serta pengambilan dan dekripsi.
- Memastikan hasil dekripsi identik dengan file asli (hash SHA-256).
- Kontrol kepemilikan data, tes unit dan integrasi.
- **Yang menyentuh bagian Anda:** memakai `owner_id` dan fungsi cek kepemilikan dari CP7, folder `instance/storage`, dan batas 50 MB.

Anggota 4: Antarmuka, Eksperimen, dan Laporan

- Halaman registrasi, login, dashboard, form data pribadi, unggah dan unduh.
- Integrasi frontend dan backend.
- Skrip benchmark bersama Anggota 2, hasil pengukuran nyata (CSV, tabel, grafik).
- Draf laporan akhir dua halaman.
- **Yang menyentuh bagian Anda:** memakai fungsi auth dan konstanta panjang username/password dari CP5, serta menyertakan token CSRF di semua form (CP6).

Note: A small overlay partly covers "menyertakan" in the last line; the wording above is inferred from the visible text.

English translation

Member 1: Backend, Authentication, Database (You)

- Flask application, configuration, SQLite and SQLAlchemy, and database models.
- Registration, login, logout, and session management.
- Authentication and authorization checks.
- Authentication and database tests.
- **Deliverables:** application foundation, authentication module, database models, data structure documentation, and tests.
- **Status:** CP1 through CP5 are complete. CP6 (routes, sessions, CSRF) is being planned. CP7 (authorization) through CP9 are not yet complete.

Member 2: Cryptography Engine

- Implement AES, DES, and RC4 using tested libraries.
- Choose non-ECB operating modes (CBC, CFB, OFB, CTR) and explain the rationale.
- Provide consistent encryption and decryption interfaces.
- Handle IVs, nonces, padding, and algorithm parameters.
- Write unit tests and document the differences between algorithms.
- **Dependencies on your work:** save results to the `cipher_copy` table (algorithm, mode, IV, wrapped key), and potentially add PyCryptodome to `requirements.txt`.

Member 3: Personal Data and File Storage

- Create, read, update, and delete personal data.
- Upload ID card images, PDF/DOC/XLS files, and videos.
- Integrate encryption with storage, retrieval, and decryption.
- Verify that decrypted files are identical to the originals using SHA-256 hashes.
- Implement data ownership controls, unit tests, and integration tests.
- **Dependencies on your work:** use `owner_id` and the ownership-checking function from CP7, the `instance/storage` folder, and the 50 MB limit.

Member 4: User Interface, Experiments, and Report

- Registration, login, dashboard, personal data forms, and upload/download pages.
- Integrate the frontend and backend.
- Develop benchmark scripts with Member 2 and produce actual measurement results (CSV, tables, charts).
- Draft a two-page final report.
- **Dependencies on your work:** use the authentication functions and username/password length constants from CP5, and include CSRF tokens in all forms (CP6).

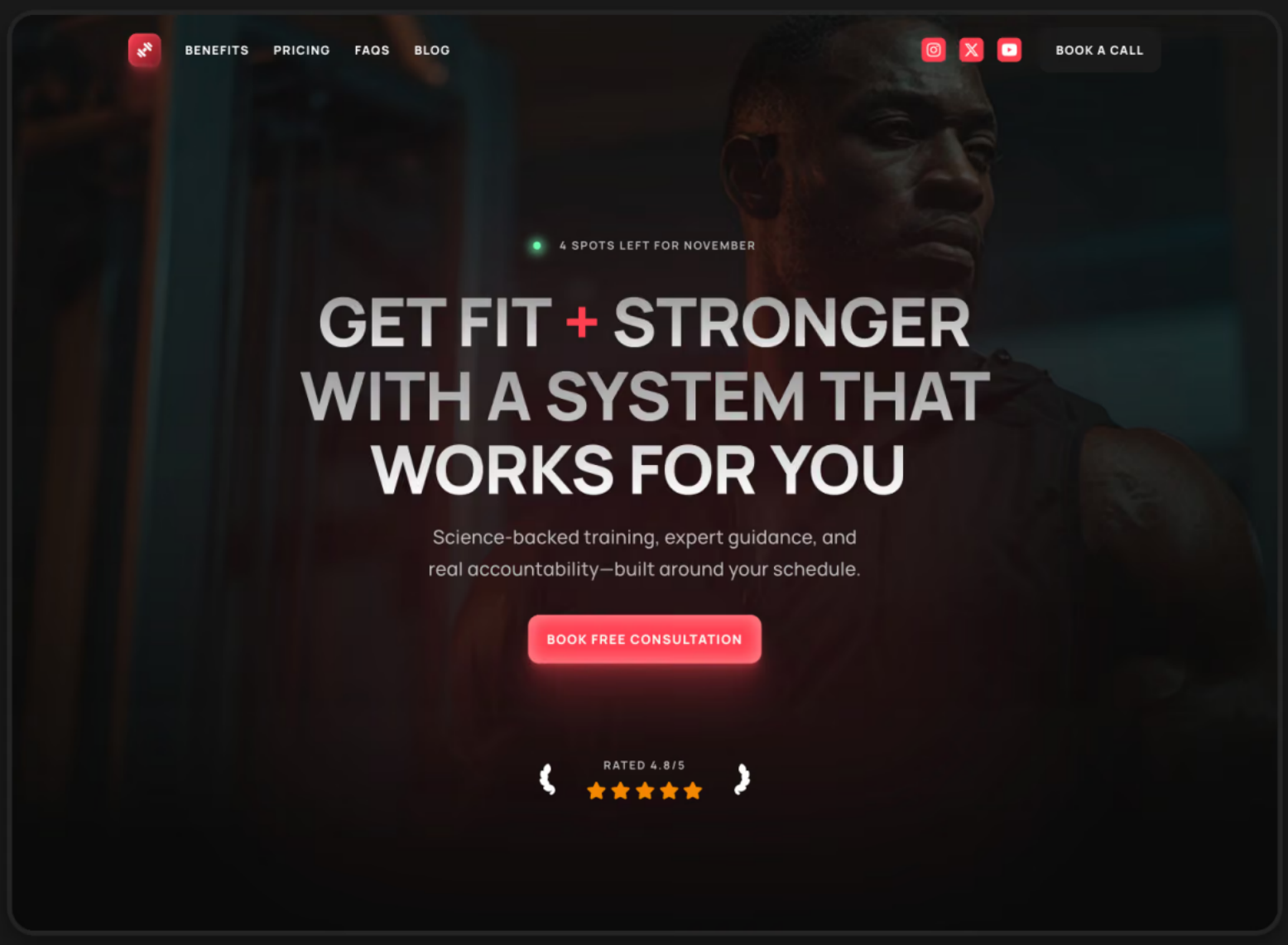
YOU / 27 Oct 2026 · 16:33

okay for this project i am taking the role of member 4. where my job is **Member 4: User Interface, Experiments, and Report**

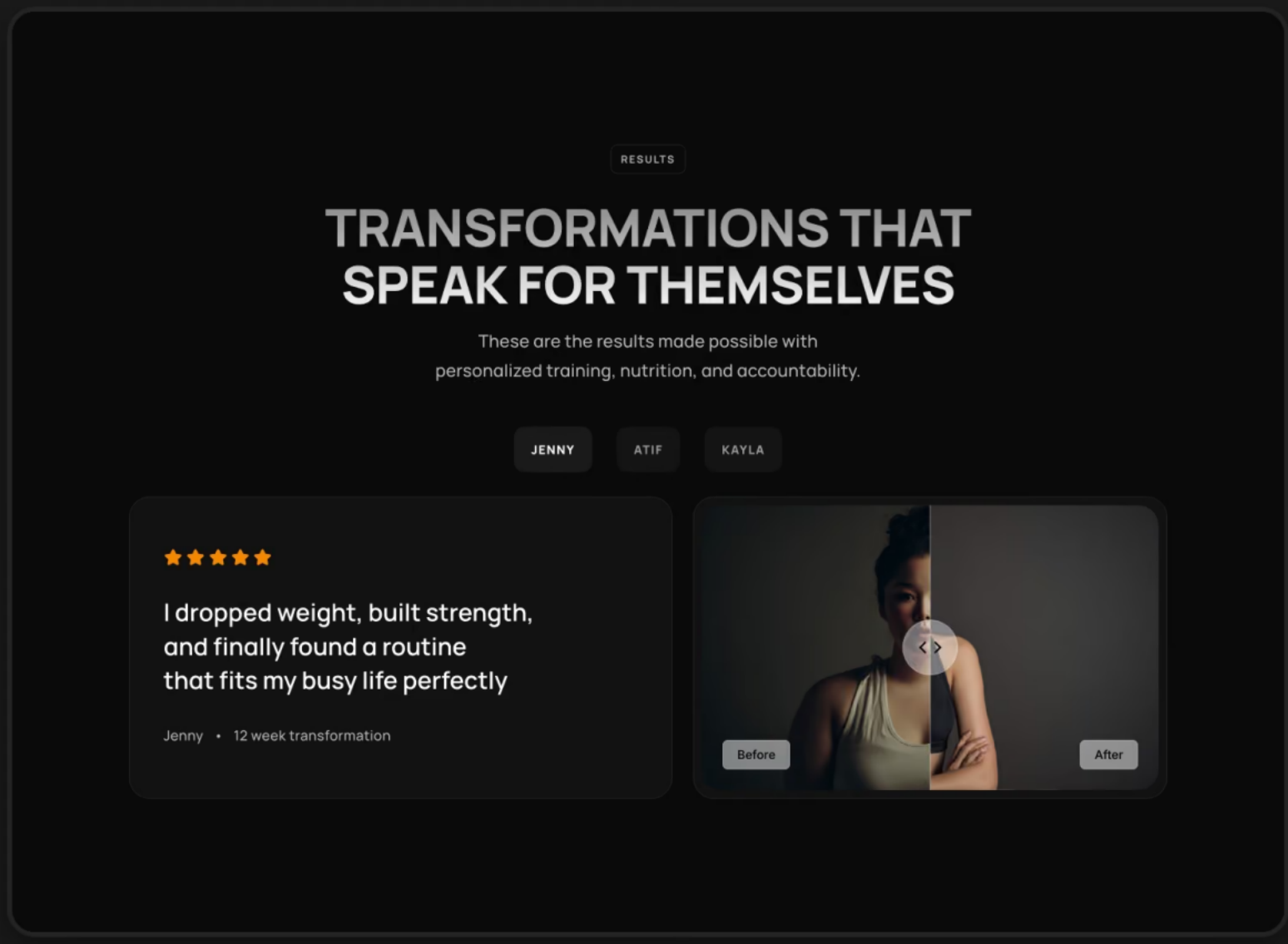
- Registration, login, dashboard, personal data forms, and upload/download pages.
- Integrate the frontend and backend.
- Develop benchmark scripts with Member 2 and produce actual measurement results (CSV, tables, charts).
- Draft a two-page final report.
- **Dependencies on your work:** use the authentication functions and username/password length constants from CP5, and include CSRF tokens in all forms (CP6).

dont think about the report and benchmark scripts for now, focus on assisting me with building the frontend of this project then integrating it with the implemented authentication and encryption and data crud. first create a one-to-one replica description for design.md based on the web design inspirations i attached, instead of a mainly red secondary palette make it blue/teal/green (which suits better). and instead of a gym theme make it medical.

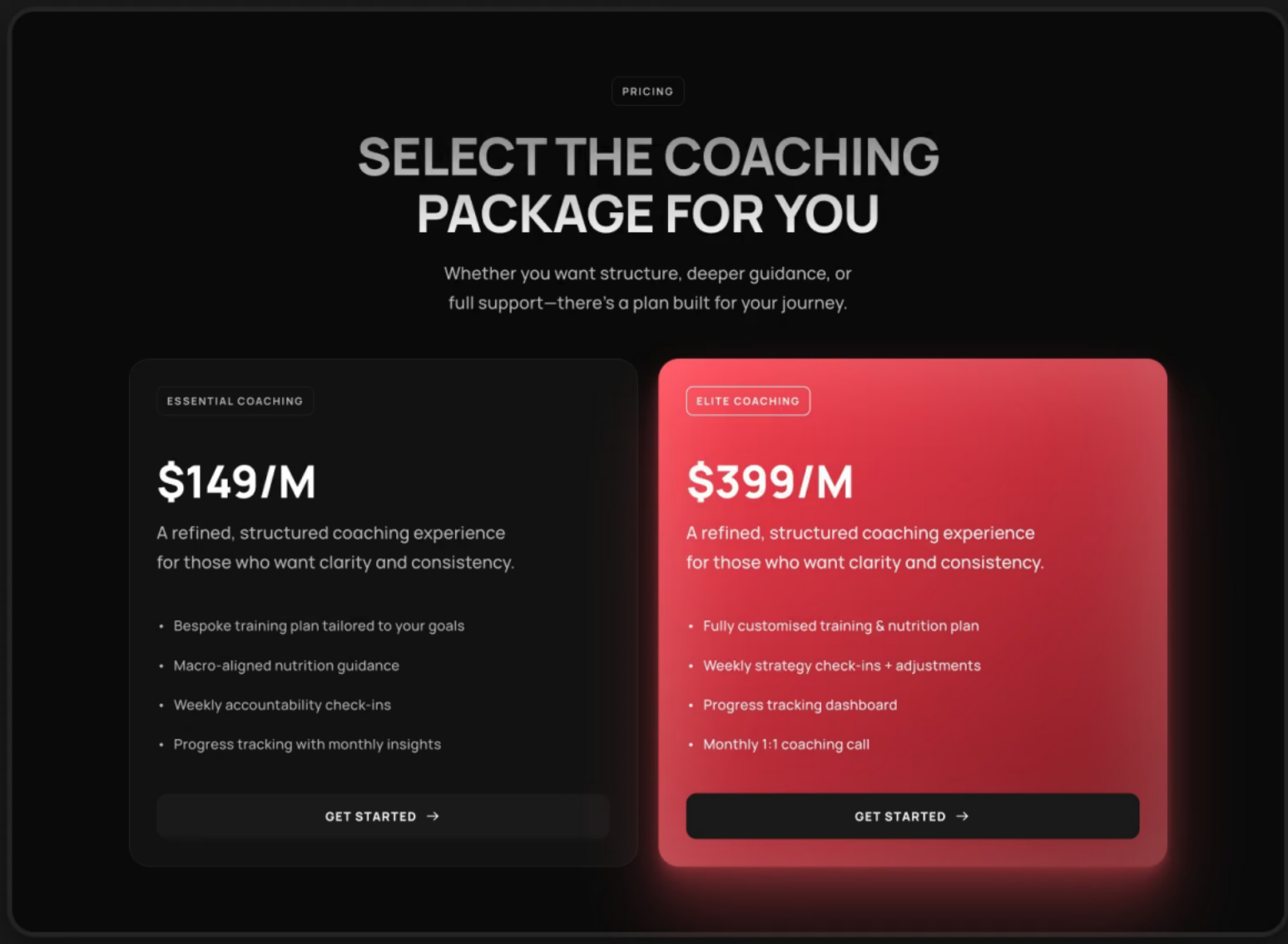
Attached reference image



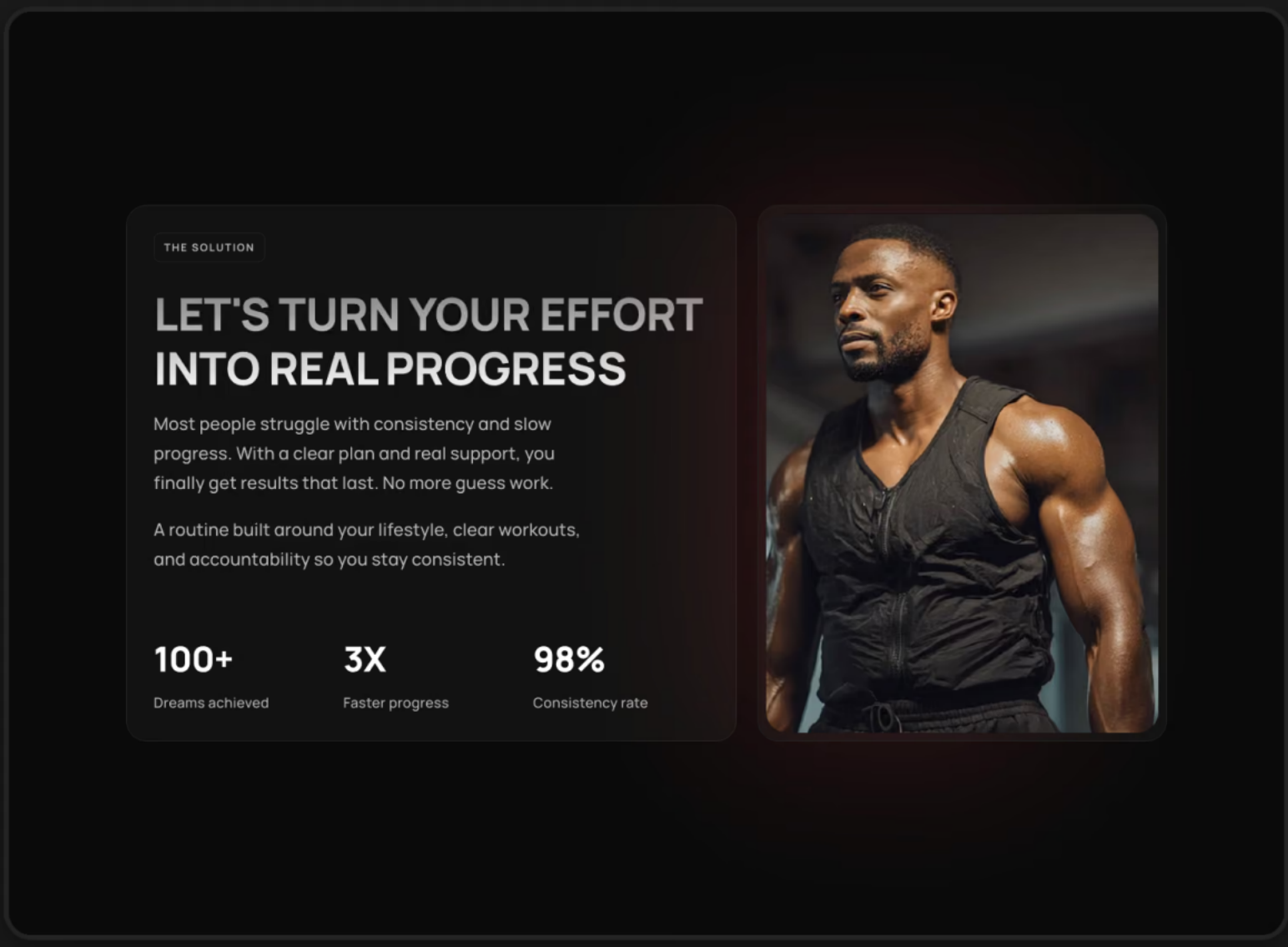
Attached reference image



Attached reference image



Attached reference image



CODEX · progress update / 27 Oct 2026 · 16:33

I'll inspect the project, then plan `design.md` around the references' layouts, typography, and effects, adapted to a medical theme with teal accents. This turn is in Plan mode, so I'll leave the file unchanged for now.

CODEX · progress update / 27 Oct 2026 · 16:34

Authentication, encryption, and storage CRUD are already present, along with a basic storage test interface; the README is behind the code. The personal-data model supports name, NIK, address, and date of birth, so a medical-themed personal records vault fits the current functionality.

The references share a clear style: near-black backgrounds, a rounded outer frame, large uppercase headings, subtle borders, generous spacing, photography, and glowing accents. I'll preserve those details in the specification.

DESIGN CHOICES / 27 Oct 2026 · 16:36

How should the medical theme relate to the existing encrypted storage app?

Your selection: Personal health vault (Recommended)

What should the initial design.md describe?

Your selection: Landing + app screens (Recommended)

Which language should the proposed interface copy use?

Your selection: English (Recommended)

CODEX / 27 Oct 2026 · 16:37

Medical Vault `design.md` Specification

Summary

Create a root-level `design.md` describing an English-language **personal health vault**, provisionally named **MedVault**. Cover the landing page, registration, login, dashboard, personal-data forms, upload, and download flows.

Preserve the references' composition, dark surfaces, typography hierarchy, spacing, rounded geometry, and lighting. Replace fitness content and red accents with medical imagery and teal accents. This step produces the design document only; frontend implementation, benchmarks, and the report remain outside this step.

Reference-Faithful Visual System

Document measurements as screenshot-derived approximations rather than verified source CSS.

- **Frame:** At the 1600 × 1200 reference size, use a charcoal canvas with approximately 60px outer margins; near-black content sits inside a 5px dark border with 32px corners and a broad, soft shadow. For the website, use one continuous frame rather than repeating the screenshot border around every section.
- **Content:** Center a 1200px maximum-width container. Use generous 120–160px section padding, 24px column gaps, 24px card corners, and 32–40px card padding.
- **Typography:** Specify locally hosted Inter with a system sans-serif fallback; the original font cannot be confirmed from images. Desktop hero text: 76px/1.12, weight 700, uppercase, tight tracking. Section headings: 60px/1.1; card headings: 48px/1.2; body: 20px/1.65; labels: 12–14px, uppercase, letter-spaced. Reserve uppercase for display headings, navigation, badges, and buttons.
- **Palette:** Canvas `#191B1B`, main background `#090C0D`, cards `#111718`, borders `#253033`, primary text `#F4F7F7`, muted text `#B7C3C5`. Teal `#2DD4BF` replaces red; blue `#38BDF8` provides supporting accents; green `#4ADE80` indicates success.
- **Effects:** Use restrained teal radial glows, inset highlights, and teal-to-deep-teal gradients. Match the references' dark overlays and soft illumination. Primary teal buttons use dark text for contrast; destructive actions retain a distinct red treatment.
- **Assets:** Specify a darkened medical-professional photograph for the hero and a portrait-oriented medical photograph for the split section. Use decorative imagery only outside private record content, served locally. Use a simple pulse-line brand mark.

Landing Page Composition

Describe each reference separately, including geometry, replacement copy, imagery, and behavior.

Reference	Preserved composition	Medical vault adaptation
1 — Hero	Transparent navigation, full-width background photograph, centered status label, oversized three-line headline, supporting text, glowing CTA, bottom reassurance strip	Navigation: Features, How It Works, FAQ; Log In and Create Account actions. Headline: **"YOUR HEALTH + RECORDS / IN ONE PLACE / UNDER YOUR CONTROL."** Supporting copy explains personal data and encrypted file storage. CTA: **Create Your Vault**. Replace ratings and scarcity messaging with factual labels: Personal Data, Encrypted Files, Owner-Only Access.
2 — Results	Centered badge and two-line heading, supporting copy, three tabs, text card beside a visual card	Badge: **YOUR RECORDS**. Heading: **"PERSONAL INFORMATION. / CLEARLY ORGANIZED."** Tabs: Personal Data, Documents, Identity & Video. Left card explains the selected category; right card shows a clearly labeled fictional record preview. Preserve the split-panel appearance without inventing a before/after health comparison or testimonial.

Reference	Preserved composition	Medical vault adaptation
3 — Pricing	Centered heading above two equally sized cards; dark left card and glowing accent right card; bottom-aligned actions	Heading: **"EVERY RECORD. / A CLEAR PLACE."** Cards: **Personal Details** and **Health Documents**. Replace prices with large category titles. Explain supported personal fields and file formats. Buttons lead to registration for visitors and the corresponding app flow for signed-in users.
4 — Solution	Approximately 60/40 text-and-image split, outlined eyebrow badge, two-line headline, paragraphs, three bottom highlights	Badge: **HOW IT WORKS**. Heading: **"KEEP YOUR RECORDS / WITHIN REACH."** Explain account creation, storage, and retrieval. Bottom highlights: **01 Create**, **02 Store**, **03 Retrieve**. Preserve the portrait crop and subtle glow.

Use the landing order **hero → solution → record categories → storage options**, with explicit labels connecting each section to its reference. Include a compact FAQ and footer using the same visual system.

Do not introduce pricing, appointments, fabricated metrics, clinical claims, or unsupported compliance claims.

Application Screens and Integration Boundaries

Carry the same dark frame, typography, cards, teal buttons, and spacing into practical application layouts.

- **Login and registration:** Two-column desktop composition with a medical image panel and a focused form card. Reuse existing authentication fields, validation, and redirects. Registration includes username, password, and password confirmation; login includes username and password. Specify inline errors, flash messages, pending states, and password visibility controls.

- **Dashboard:** Compact top navigation with Dashboard, Personal Data, Files, and POST logout. Show actual record counts and total original-file size, primary actions for Add Personal Data and Upload File, and a recent-record list. Include first-use, loading, error, and populated states.

- **Personal data:** List, view, create, edit, and delete records containing full name, NIK, address, and date of birth. Preserve API field names `nama`, `nik`, `alamat`, and `tanggal_lahir`. Mask NIK in summaries and reveal full details only in the record view.

- **Files:** Category filters, filename, category, original size, and download/delete controls. Upload accepts identity images, supported documents, or MP4 according to existing validation. Provide a native file picker alongside the styled drop zone.

- **Downloads:** Offer AES, DES, and RC4 copy selection, defaulting to AES. Explain that downloading decrypts the selected copy and returns the original file. Identify DES and RC4 as legacy algorithms included for the coursework.

- **Feedback:** Specify accessible deletion confirmation, retryable errors, expired-session handling, upload validation, and honest pending states. Do not display simulated upload percentages.

Document the existing storage endpoints and payloads as integration contracts. Preserve Flask/Jinja, existing authentication services and length constants, ownership checks, encryption services, and API behavior. Use CSRF hidden fields for server forms and `X-CSRFToken` for JavaScript mutations. CSS, JavaScript, fonts, and images must be served locally under the existing CSP; private data must not enter browser persistent storage.

Responsive Behavior and Acceptance

- At widths below 1024px, reduce display typography and spacing. Below 768px, stack columns, collapse navigation into an accessible menu, make primary actions full-width, and convert record tables into readable cards.

- At 375px, use approximately 16px outer spacing, 20px card padding, 36–40px hero text, and no horizontal overflow.

- Specify visible keyboard focus, associated form labels, accessible tabs and dialogs, text accompanying status colors, 44px minimum touch targets, and reduced-motion support.

- Include a later implementation checklist covering visual comparison at 1600px, tablet and mobile layouts, registration/login/logout, personal-data CRUD, upload, each download algorithm, deletion, CSRF errors, and session expiry.

- Acceptance for this documentation step: every reference has a concrete adaptation; all requested app screens and major states are described; all proposed fields and actions map to existing backend capabilities.

Assumptions: English interface, personal health vault positioning, provisional MedVault branding, teal as the dominant accent, and fictional data for demonstrations. Preserve existing uncommitted work when implementation begins.

YOU / 27 Oct 2026 · 16:37

PLEASE IMPLEMENT THIS PLAN:

Medical Vault `design.md` Specification

Summary

Create a root-level `design.md` describing an English-language **personal health vault**, provisionally named **MedVault**. Cover the landing page, registration, login, dashboard, personal-data forms, upload, and download flows.

Preserve the references' composition, dark surfaces, typography hierarchy, spacing, rounded geometry, and lighting. Replace fitness content and red accents with medical imagery and teal accents. This step produces the design document only; frontend implementation, benchmarks, and the report remain outside this step.

Reference-Faithful Visual System

Document measurements as screenshot-derived approximations rather than verified source CSS.

- **Frame:** At the 1600 × 1200 reference size, use a charcoal canvas with approximately 60px outer margins; near-black content sits inside a 5px dark border with 32px corners and a broad, soft shadow. For the website, use one continuous frame rather than repeating the screenshot border around every section.
- **Content:** Center a 1200px maximum-width container. Use generous 120–160px section padding, 24px column gaps, 24px card corners, and 32–40px card padding.
- **Typography:** Specify locally hosted Inter with a system sans-serif fallback; the original font cannot be confirmed from images. Desktop hero text: 76px/1.12, weight 700, uppercase, tight tracking. Section headings: 60px/1.1; card headings: 48px/1.2; body: 20px/1.65; labels: 12–14px, uppercase, letter-spaced. Reserve uppercase for display headings, navigation, badges, and buttons.
- **Palette:** Canvas `#191B1B`, main background `#090C0D`, cards `#111718`, borders `#253033`, primary text `#F4F7F7`, muted text `#B7C3C5`. Teal `#2DD4BF` replaces red; blue `#38BDF8` provides supporting accents; green `#4ADE80` indicates success.
- **Effects:** Use restrained teal radial glows, inset highlights, and teal-to-deep-teal gradients. Match the references' dark overlays and soft illumination. Primary teal buttons use dark text for contrast; destructive actions retain a distinct red treatment.
- **Assets:** Specify a darkened medical-professional photograph for the hero and a portrait-oriented medical photograph for the split section. Use decorative imagery only outside private record content, served locally. Use a simple pulse-line brand mark.

Landing Page Composition

Describe each reference separately, including geometry, replacement copy, imagery, and behavior.

Reference	Preserved composition	Medical vault adaptation
1 — Hero	Transparent navigation, full-width background photograph, centered status label, oversized three-line headline, supporting text, glowing CTA, bottom reassurance strip	Navigation: Features, How It Works, FAQ; Log In and Create Account actions. Headline: **"YOUR HEALTH + RECORDS / IN ONE PLACE / UNDER YOUR CONTROL."** Supporting copy explains personal data and encrypted file storage. CTA: **Create Your Vault**. Replace ratings and scarcity messaging with factual labels: Personal Data, Encrypted Files, Owner-Only Access.
2 — Results	Centered badge and two-line heading, supporting copy, three tabs, text card beside a visual card	Badge: **YOUR RECORDS**. Heading: **"PERSONAL INFORMATION. / CLEARLY ORGANIZED."** Tabs: Personal Data, Documents, Identity & Video. Left card explains the selected category; right card shows a clearly labeled fictional record preview. Preserve the split-panel appearance without inventing a before/after health comparison or testimonial.

Reference	Preserved composition	Medical vault adaptation
3 — Pricing	Centered heading above two equally sized cards; dark left card and glowing accent right card; bottom-aligned actions	Heading: **"EVERY RECORD. / A CLEAR PLACE."** Cards: **Personal Details** and **Health Documents**. Replace prices with large category titles. Explain supported personal fields and file formats. Buttons lead to registration for visitors and the corresponding app flow for signed-in users.
4 — Solution	Approximately 60/40 text-and-image split, outlined eyebrow badge, two-line headline, paragraphs, three bottom highlights	Badge: **HOW IT WORKS**. Heading: **"KEEP YOUR RECORDS / WITHIN REACH."** Explain account creation, storage, and retrieval. Bottom highlights: **01 Create**, **02 Store**, **03 Retrieve**. Preserve the portrait crop and subtle glow.

Use the landing order **hero → solution → record categories → storage options**, with explicit labels connecting each section to its reference. Include a compact FAQ and footer using the same visual system.

Do not introduce pricing, appointments, fabricated metrics, clinical claims, or unsupported compliance claims.

Application Screens and Integration Boundaries

Carry the same dark frame, typography, cards, teal buttons, and spacing into practical application layouts.

- **Login and registration:** Two-column desktop composition with a medical image panel and a focused form card. Reuse existing authentication fields, validation, and redirects. Registration includes username, password, and password confirmation; login includes username and password. Specify inline errors, flash messages, pending states, and password visibility controls.

- **Dashboard:** Compact top navigation with Dashboard, Personal Data, Files, and POST logout. Show actual record counts and total original-file size, primary actions for Add Personal Data and Upload File, and a recent-record list. Include first-use, loading, error, and populated states.

- **Personal data:** List, view, create, edit, and delete records containing full name, NIK, address, and date of birth. Preserve API field names `nama`, `nik`, `alamat`, and `tanggal_lahir`. Mask NIK in summaries and reveal full details only in the record view.

- **Files:** Category filters, filename, category, original size, and download/delete controls. Upload accepts identity images, supported documents, or MP4 according to existing validation. Provide a native file picker alongside the styled drop zone.

- **Downloads:** Offer AES, DES, and RC4 copy selection, defaulting to AES. Explain that downloading decrypts the selected copy and returns the original file. Identify DES and RC4 as legacy algorithms included for the coursework.

- **Feedback:** Specify accessible deletion confirmation, retryable errors, expired-session handling, upload validation, and honest pending states. Do not display simulated upload percentages.

Document the existing storage endpoints and payloads as integration contracts. Preserve Flask/Jinja, existing authentication services and length constants, ownership checks, encryption services, and API behavior. Use CSRF hidden fields for server forms and `X-CSRFToken` for JavaScript mutations. CSS, JavaScript, fonts, and images must be served locally under the existing CSP; private data must not enter browser persistent storage.

Responsive Behavior and Acceptance

- At widths below 1024px, reduce display typography and spacing. Below 768px, stack columns, collapse navigation into an accessible menu, make primary actions full-width, and convert record tables into readable cards.

- At 375px, use approximately 16px outer spacing, 20px card padding, 36–40px hero text, and no horizontal overflow.

- Specify visible keyboard focus, associated form labels, accessible tabs and dialogs, text accompanying status colors, 44px minimum touch targets, and reduced-motion support.

- Include a later implementation checklist covering visual comparison at 1600px, tablet and mobile layouts, registration/login/logout, personal-data CRUD, upload, each download algorithm, deletion, CSRF errors, and session expiry.

- Acceptance for this documentation step: every reference has a concrete adaptation; all requested app screens and major states are described; all proposed fields and actions map to existing backend capabilities.

Assumptions: English interface, personal health vault positioning, provisional MedVault branding, teal as the dominant accent, and fictional data for demonstrations. Preserve existing uncommitted work when implementation begins.

CODEX · progress update / 27 Oct 2026 · 16:37

I'll create the root-level `design.md` with the visual specifications, screen layouts, and verified backend contracts. I'll leave the application code and existing uncommitted changes untouched.

CODEX · progress update / 27 Oct 2026 · 16:41

The document is written, including all four reference adaptations and the application screens. It also accounts for backend details that affect the UI: the record list omits decrypted personal fields, downloads offer all three algorithm copies, and upload limits include multipart overhead. I'm checking the file and confirming that no application files changed.

CODEX / 27 Oct 2026 · 16:41

Created design.md with all four reference adaptations, the medical teal visual system, application screens, backend contracts, and responsive/accessibility requirements.

Existing application changes were preserved. Frontend implementation remains outside this documentation step.

YOU / 27 Oct 2026 · 16:47

aight, now I want to build the frontend for this project using `design.md` as the main design reference. Please read it first, then inspect the existing Flask templates, routes, and storage API so you understand how the frontend should connect to the backend. Let's work one page or component at a time so I can run it and check each change before we continue (give a guide on how to run and check it as well).
Start by briefly explaining the design direction and suggesting a sensible order for building the pages. Then implement only the first page in that order and stop for me to review it.
Use the existing Flask and Jinja setup. Keep authentication, encryption, ownership checks, and CSRF protection working. Check the existing Content Security Policy before adding CSS or JavaScript, and use separate static files where required.
The application is a private medical referral workspace. Patients manage their personal information, identity scans, medical documents, and uploaded symptom videos. Match the features that the backend actually supports, and tell me if the design calls for something that needs additional backend work.
After each step, explain what changed, tell me how to open and test it, and list a few things I should check. Wait for my feedback before starting the next page. Use simple language when explaining technical decisions.

CODEX · progress update / 27 Oct 2026 · 16:47

The design points toward a dark, calm medical workspace: large headings, rounded panels, medical imagery, and teal highlights. I'd build the public landing page first, then registration and login, followed by the dashboard, personal information, and file upload/download screens. I'll inspect the existing code and implement only the landing page for your review. I'll use the Sites skill as guidance for the visual work.

CODEX · progress update / 27 Oct 2026 · 16:47

The current home page doubles as a working storage test after login, so I'll keep that authenticated view available while replacing only the public landing page. The existing security policy requires local styles and scripts; I'll use separate static files and keep the existing login, registration, and logout routes intact.

CODEX · progress update / 27 Oct 2026 · 16:47

I'll use the imagegen skill to create a local medical photograph for the landing page, matching the reference's dark, portrait-led look. The current backend supports storing records and videos, but it does not yet provide referral sharing between patients and clinicians, so the page won't claim that feature exists.

CODEX · progress update / 27 Oct 2026 · 16:50

The landing page now has the four reference sections, responsive navigation, record-category tabs, and links to the existing account pages. The authenticated storage test remains in place; I'm checking the new page against the existing tests and its security policy now.

CODEX · generated image / 27 Oct 2026 · 16:50

Generated website image



CODEX · progress update / 27 Oct 2026 · 16:51

The page and its local CSS, JavaScript, and image all return successfully under the existing security policy. The first test run hit a Windows temporary-folder permission problem, so I'm rerunning the focused checks with a temporary folder inside the project.

CODEX · progress update / 27 Oct 2026 · 16:53

The focused checks found one outdated test: it assumes the public home page has no JavaScript at all. The new tabs use a local script allowed by the existing policy, so I'm updating that check to require the specific local script while keeping the authentication pages script-free.

CODEX · progress update / 27 Oct 2026 · 16:55

The landing page is ready to review at the local server address. The preview panel couldn't attach in this session, but the server itself is running; I'm finishing the regression checks and cleaning up the temporary test files.

CODEX / 27 Oct 2026 · 16:55

The first page is ready: a public MedVault landing page with the dark teal design, medical imagery, record-category tabs, FAQ, and links to the existing login and registration pages.

The signed-in storage test page is still available after login. The new CSS, JavaScript, and image are served locally, in line with the existing security policy. **All 231 focused authentication, storage, and security tests passed.** I couldn't complete a visual check in the preview panel because it failed to attach, so your browser review will be useful.

The local server is running at http://127.0.0.1:8000 (http://127.0.0.1:8000). If you need to start it again, open PowerShell in the project folder and run:

POWERSHELL

```
.\.venv\Scripts\python.exe run.py
```

To review:

1. Open the page while logged out and scroll through each section.
2. Try the record-category tabs, FAQ, navigation links, and account buttons.
3. Narrow your browser window to check the mobile layout and menu.
4. Log in and confirm the existing storage test page still appears.

One backend gap for the referral-workspace goal: patients can store and retrieve their own records, but sharing records or referrals with clinicians would require additional backend work.

I'll wait for your feedback before starting the next page.

YOU / 27 Oct 2026 · 16:59

i get this error. This site can't be reached

127.0.0.1 took too long to respond.

Try:

- Checking the connection
- Checking the proxy and the firewall

ERR_CONNECTION_TIMED_OUT

CODEX · progress update / 27 Oct 2026 · 16:59

I'll check whether the Flask server is still running and whether it responds on port 8000, then give you the exact steps to open it from your browser.

CODEX · progress update / 27 Oct 2026 · 16:59

The Flask process I started is still running, but it isn't reachable from outside its sandbox. I'll start the server where your browser can reach it.

CODEX / 27 Oct 2026 · 17:00

The server I started wasn't reachable from your browser, even after I retried outside the sandbox. I've stopped those server processes so they won't occupy the port.

Please start it in your own VS Code terminal:

1. Open **Terminal → New Terminal**.
2. Run:

POWERSHELL

```
cd C:\vscodeProjects\ki-tugas1\ki-assignment-1
.\.venv\Scripts\python.exe run.py
```

3. Keep the terminal open. Once it says `Running on http://127.0.0.1:8000`, open http://127.0.0.1:8000 (http://127.0.0.1:8000) in your browser.

If it still doesn't open, send me the terminal output so I can identify what's preventing it from starting.

YOU / 27 Oct 2026 · 17:09

works now for some reason. Please continue building the remaining frontend pages using `design.md` and the landing page's visual style. First, check which pages are already complete and identify the remaining pages supported by the backend. These may include registration, login, the patient dashboard, personal-data creation and editing, file uploads, and the list of stored records with download and delete controls. Work on **one page at a time**. Start with the next unfinished page in the user journey. After implementing it, explain what changed and give me the URL and steps to test it. **Stop and wait for my feedback before continuing to another page.** Connect the pages to the actual backend. Include clear loading, success, error, and empty states where needed. Make the layouts work on both desktop and mobile. Preserve authentication, ownership checks, CSRF protection, and the existing encryption behavior. Follow the project's Content Security Policy when adding styles and scripts. Use patient-friendly labels for the medical referral scenario. Keep technical encryption details out of ordinary patient tasks unless they are needed for the assignment's comparison or download controls. If a feature needs backend work, explain the gap before implementing it. Also add a compact project-credits section to the landing page footer:

Information Security — Assignment 1

Group 2

| Name | NRP |

| ---------- | ---------- |

| Fauzan Hafiz Amandani | 5025241087 |

| Farrel Aqilla Novianto | 5025241015 |

| Ibrahim Ferel | 5025241049 |

| Fayza Lathifah Humam | 5025241094 |

Keep the credits consistent with the design and readable on mobile.

I'll run the application from my own VS Code terminal. Tell me when I need to restart the server or refresh the browser. Explain technical decisions in simple language.

CODEX · progress update / 27 Oct 2026 · 17:09

The landing page is in place. The next step in the patient journey is registration, followed by login, the dashboard, personal details, and stored files. I'll connect the registration design to the existing account form and add the requested footer credits, then stop for your review.

CODEX · progress update / 27 Oct 2026 · 17:09

Registration already has a working backend, including account rules, duplicate-name checks, CSRF protection, and a success redirect to login. I can build this page without backend changes. The later referral-sharing feature still needs backend support; the existing storage API currently keeps records accessible only to their owner.

CODEX · progress update / 27 Oct 2026 · 17:11

The registration page now uses the existing Flask form and account rules, with a responsive layout, password visibility controls, and clear validation messages. I've also added the requested credits to the landing-page footer. I'm checking registration, CSRF, and the existing authentication tests before handing it back.

CODEX · progress update / 27 Oct 2026 · 17:12

The first test run exposed two compatibility issues: the existing rate-limit test expects a specific alert format, and a Windows test reads templates using the system encoding. I've adjusted the template to preserve the existing alert and use portable character entities; I'm rerunning the checks.

CODEX · progress update / 27 Oct 2026 · 17:13

All 231 checks passed on the rerun, but pytest reported an unclosed SQLite connection during shutdown and returned an error code. I'm checking once more with that cleanup warning suppressed so I can distinguish the page checks from the test environment's shutdown issue.

CODEX / 27 Oct 2026 · 17:14

The registration page is ready for review. It now matches the landing page's dark teal style, adapts to mobile screens, and uses the existing account validation and CSRF protection. I also added the Group 2 credits to the landing-page footer.

Restart your server to load the updated templates: press `Ctrl+C` in your VS Code terminal, then run:

POWERSHELL

```
.\.venv\Scripts\python.exe run.py
```

Open http://127.0.0.1:8000/auth/register (http://127.0.0.1:8000/auth/register). If you're already logged in, log out first.

Please check:

- The layout on desktop and a narrow browser window.
- The password Show/Hide buttons and validation messages, including mismatched passwords.
- Creating a test account: success should take you to the existing login page.
- The credits at the bottom of the landing page (http://127.0.0.1:8000/).

The login page still has its earlier appearance; it's the next unfinished page. The authentication and security checks passed: **231 tests**. One run reported an SQLite cleanup warning at shutdown; the repeat run passed with that warning suppressed.

I'll wait for your feedback before continuing.

YOU / 27 Oct 2026 · 17:18

okayyit works, please move on to the login page, then the main file-upload and records page, following design.md and the landing page's visual style.

Implement the login page first. Connect it to the existing authentication route, include clear validation and error messages, and send users to the main upload page after successful login. Keep CSRF protection and session handling working. Give me the steps to test it, then stop and wait for my feedback.

After I approve the login page, build the main upload page. It should let the logged-in patient upload identity scans, medical documents, and symptom videos, view their own stored items, download them, and delete them with confirmation. Show accepted formats, file-size limits, upload progress or a loading state, and useful success and error messages.

Make the AES, DES, and RC4 comparison a clear part of this page. First inspect what measurements the backend actually exposes. For a selected uploaded item, show a comparison table with each algorithm's mode, encryption time, encrypted size, and storage overhead where available. Provide separate download controls for recovering that item from its AES, DES, and RC4 copies.

Explain the measurements in simple language. Clearly label whether a time measures just encryption or the complete upload process. Show units and explain how overhead is calculated. Never invent measurements or use browser request time as cipher-processing time. If the backend does not expose a metric, identify the gap and propose the smallest backend change needed.

Label these as measurements from an individual upload, rather than a complete benchmark. Include a brief note that DES and RC4 are included for the assignment's educational comparison and that speed alone does not establish security. Use fictional data in examples.

Keep the page responsive and accessible, preserve ownership checks and the existing cryptography, and follow the Content Security Policy when adding CSS and JavaScript. Work in small steps, explain what changed, and wait for my verification before moving to the next step.

CODEX · progress update / 27 Oct 2026 · 17:18

I'll implement the login page first and keep its existing authentication and session handling. A successful login already leads to `/`, which currently contains the signed-in storage test; that will serve as the temporary upload page until we build its replacement after your approval.

CODEX · progress update / 27 Oct 2026 · 17:24

The login page now matches the registration screen and uses the existing form, including its generic error for incorrect credentials. I've kept the current redirect behavior, including safe return links, and will verify login, CSRF, and session handling with the existing tests.

CODEX · progress update / 27 Oct 2026 · 17:25

The login form is connected to the existing route, and the current storage page remains the destination after a normal login. The automated checks are still running; they also cover failed login attempts and the session reset.

CODEX / 27 Oct 2026 · 17:26

The login page is ready. It matches the registration page, includes a password visibility control and clear login errors, and uses the existing authentication, CSRF, and session handling. A successful login takes you to the current signed-in storage page.

Restart the server in your VS Code terminal with `Ctrl+C`, then run:

POWERSHELL

```
.\.venv\Scripts\python.exe run.py
```

Open http://127.0.0.1:8000/auth/login (http://127.0.0.1:8000/auth/login). If you're already logged in, log out first.

Please check the desktop and mobile layouts, Show/Hide password, an incorrect password, and a successful login. The current destination still says **Storage test**; we'll replace it with the designed upload and records page in the next step.

The authentication and security checks passed: **231 tests**, with the previously observed SQLite cleanup warning suppressed. I'll wait for your feedback before building the upload page or its comparison table.

YOU / 27 Oct 2026 · 17:30

okay continue with the main storage page, remember the previous instructions about encryption method comparisons.

CODEX · progress update / 27 Oct 2026 · 17:30

I'll inspect the storage and encryption code first to see which comparison measurements are already recorded and which are available to the page. Then I'll build the signed-in storage page around the existing upload, ownership, download, and delete behavior.

CODEX · progress update / 27 Oct 2026 · 17:31

The backend already saves each copy's algorithm, mode, encryption time, and encrypted size, but its API doesn't expose those measurements. I'll add a small owner-checked read endpoint for the selected item; it will return only the comparison fields and original size, with no keys or storage paths. The recorded time covers the cipher operation only, while encrypted size includes the authentication tag.

CODEX · progress update / 27 Oct 2026 · 17:34

The comparison endpoint now reads the saved measurements through the same ownership check used for downloads. I'm building the page so patients can upload and manage files while the comparison stays clearly labeled as a single-record coursework measurement.

CODEX · progress update / 27 Oct 2026 · 17:35

The page now connects uploads, personal records, deletion, and downloads to the existing backend. The comparison calculates overhead from the saved ciphertext size and original size, and labels the timing as the cipher operation rather than the whole upload. I'm running the storage, encryption, ownership, and security checks now.

CODEX · progress update / 27 Oct 2026 · 17:37

The focused checks passed, including byte-for-byte recovery through all three algorithms and a test that another user cannot access the comparison. I'm finishing a final check of the page's loading and confirmation behavior; you'll need to restart your server because the comparison adds a backend route.

CODEX / 27 Oct 2026 · 17:37

The main storage page is ready. It supports uploading identity scans, medical documents, and symptom videos; viewing your records; editing personal information; and deleting records with confirmation.

For each selected record, **Compare & download** shows AES, DES, and RC4 mode, encryption time, encrypted size, and calculated overhead, with a separate download for each copy. The page explains that the time covers the cipher operation, and that these are measurements for one record rather than a complete benchmark.

I added a small backend endpoint to expose the measurements already saved for each copy. It uses the existing ownership check. The focused storage, encryption, authentication, and security checks passed: **328 tests**.

Restart the server in your VS Code terminal so it loads the new endpoint:

POWERSHELL

```
# Press Ctrl+C first, then run:
.\.venv\Scripts\python.exe run.py
```

Open http://127.0.0.1:8000 (http://127.0.0.1:8000) and log in. Please check:

1. Upload a **fictional** PDF or image and confirm it appears in Your records.
2. Select **Compare & download**, review the measurements, and download the original from each algorithm's copy.
3. Try an unsupported file type and check the error message.
4. Test the delete confirmation and the layout on a narrow browser window.
5. Confirm your existing personal-data form still works.

The page has been tested through the backend checks; I haven't visually verified it in your browser. I'll wait for your feedback before making the next change.

YOU / 27 Oct 2026 · 17:43

when the "your records" section hits 4 rows, make it scroll or go next. also remove the "
YOUR PERSONAL VAULT " tab thingy from all instances. make the name of each method in the comparison section more obvious

CODEX · progress update / 27 Oct 2026 · 17:44

I'll make the records list scroll after four rows, remove that "Your personal vault" label wherever it appears, and make the algorithm names stand out more clearly in the comparison table.

YOU / 27 Oct 2026 · 17:44

actually make the download/retrieve back the file the main focus. the comparison is an optional look/not the main focus

CODEX · progress update / 27 Oct 2026 · 17:44

I'll make downloading the original file the primary action on each record, using its AES copy. The comparison will remain available as a separate optional action, with AES, DES, and RC4 downloads still available there.

CODEX · progress update / 27 Oct 2026 · 17:45

Each record now has a prominent **Download original** action, while **Compare methods** opens the optional measurements and the three algorithm-specific downloads. The records list uses pages of four, so it stays compact on both desktop and mobile.

CODEX / 27 Oct 2026 · 17:45

Updated the storage page:

- Records appear four per page, with Previous and Next controls when needed.
- **Download original** is the main action and uses the AES copy. **Compare methods** remains optional and offers all three downloads.
- AES, DES, and RC4 names stand out in the comparison table.
- Removed the personal-vault label from the landing, login, registration, and storage pages.

Restart the Flask server, then refresh the page. Try it with at least five records to check pagination, and select **Compare methods** to review the table. The storage and authentication checks passed: **159 tests**.

YOU / 27 Oct 2026 · 17:50

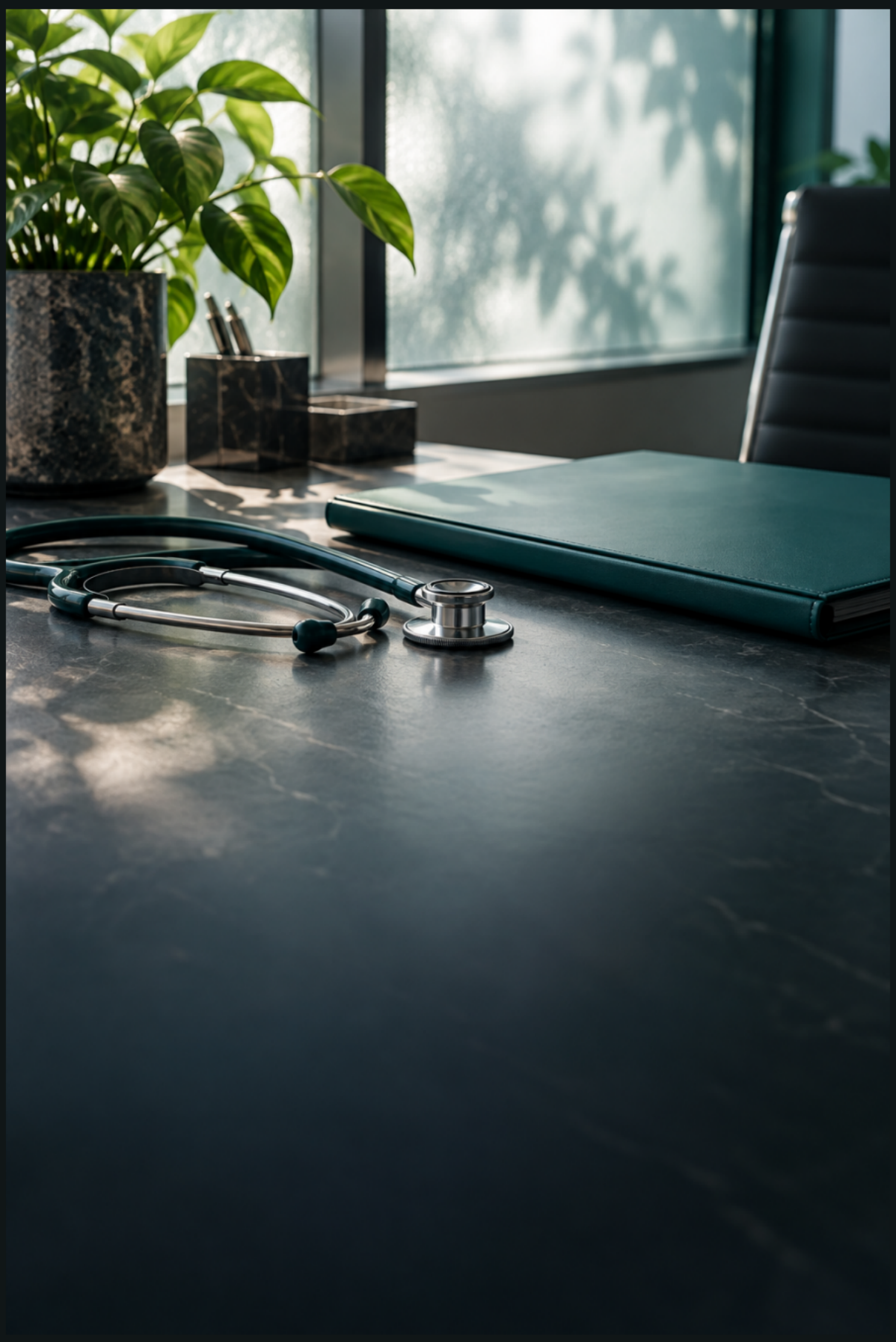
actually make it after 3 rows. create some more stock photos so it's not that one lady over and over. also no need to state "**OPTIONAL ASSIGNMENT COMPARISON" in the comparison section**

CODEX · progress update / 27 Oct 2026 · 17:51

I'll change the list to three records per page, remove the comparison badge, and use the imagegen skill to create distinct medical photos for the landing, registration, and login pages.

CODEX · generated image / 27 Oct 2026 · 17:52

Generated website image



CODEX · progress update / 27 Oct 2026 · 17:52

The list now uses three records per page, and the comparison badge is removed. The new images are being generated: a doctor portrait for the landing page, a clinical workspace for registration, and a hospital corridor scene for login.

CODEX · generated image / 27 Oct 2026 · 17:52

Generated website image



CODEX · generated image / 27 Oct 2026 · 17:52

Generated website image



CODEX / 27 Oct 2026 · 17:53

Done:

- Records now show **three per page**.
- Removed the “Optional Assignment Comparison” badge.
- Added three distinct medical images for the landing section, registration, and login.

JavaScript syntax and file checks passed. Restart Flask, then hard-refresh with **Ctrl+Shift+R** to see the updated templates and images.

YOU / 27 Oct 2026 · 17:54

how do i go back to the main branch in my local clone but keep the changes i've made

CODEX / 27 Oct 2026 · 17:55

To switch to `main` and bring your current changes with you, run:

POWERSHELL

```
git stash push -u -m "Save frontend work"
git switch main
git stash pop
git status
```

`-u` includes new files such as your CSS, JavaScript, and images. Ignored files like `.env` stay on your computer.

If `git stash pop` reports conflicts, your saved stash remains available—resolve the conflicts before continuing.

If you only want to **save the work and view a clean** `main`, stop after `git switch main`. Later, return to your original branch and run `git stash pop`.

YOU / 27 Oct 2026 · 17:55

this wont commit to main right? cause after that i want to create a new branch myself

CODEX / 27 Oct 2026 · 17:55

Correct—those commands **don't create any commits**.

Since you want a new branch, restore your changes after creating it:

POWERSHELL

```
git stash push -u -m "Save frontend work"
git switch main
git switch -c your-new-branch-name
git stash pop
```

Your changes will be uncommitted on the new branch, and `main` stays unchanged.

YOU / 27 Oct 2026 · 17:57

PS C:\vscodeProjects\ki-tugas1\ki-assignment-1> git stash push -u -m "feat: save frontend work"

git switch main

git switch -c frontend-integration

git stash pop
warning: in the working copy of 'app/static/css/credits.css', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'app/static/css/landing.css', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'app/static/css/medical-images.css', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'app/static/css/register.css', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'app/static/css/storage-refinements.css', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'app/static/css/storage.css', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'app/static/js/landing.js', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'app/static/js/login.js', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'app/static/js/register.js', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'app/static/js/storage.js', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'app/templates/landing.html', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'design.md', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'app/templates/auth/register.html', LF will be replaced by CRLF the next time Git touches it
Saved working directory and index state On feature/crud-data: feat: save frontend work
fatal: invalid reference: main

Switched to a new branch 'frontend-integration'
On branch frontend-integration
Changes not staged for commit:
(use "git add <file>..." to update what will be committed)
(use "git restore <file>..." to discard changes in working directory)
modified: app/__init__.py
modified: app/placeholder.py
modified: app/storage/routes.py
modified: app/templates/auth/login.html
modified: app/templates/auth/register.html
modified: app/templates/base.html
modified: app/templates/placeholder_home.html
modified: tests/test_auth_routes.py
modified: tests/test_storage_routes.py

Untracked files:
(use "git add <file>..." to include in what will be committed)
app/static/
app/templates/landing.html
design.md

no changes added to commit (use "git add" and/or "git commit -a")
Dropped refs/stash@{0} (a1ee74fdf4f406dbcdf5e45954b208ab0f82e8ca)
what does that mean...

CODEX / 27 Oct 2026 · 17:57

Your changes are safe, and **nothing was committed**.

Here's what happened:

1. Git saved your changes in a stash.
2. `git switch main` failed because your clone has no local branch named `main`.
3. Git still ran the next command, creating `frontend-integration` from `feature/crud-data`, the branch you were on.
4. `git stash pop` successfully restored all your changes onto `frontend-integration`, then removed the temporary stash.

You're now on `frontend-integration`, with your work uncommitted. That's fine if you want the frontend branch based on the existing CRUD implementation.

The LF/CRLF messages are Windows line-ending warnings; they don't mean your work was lost.

To see which branches actually exist, run:

POWERSHELL

```
git branch -a
```

Share that output if you still want the new branch based on the repository's main/default branch.

YOU / 27 Oct 2026 · 17:57

oh no i mean go to the main branch in the connected repo

CODEX · progress update / 27 Oct 2026 · 17:58

I'll check the remote branches and repository's default branch so we can move your frontend branch to the right starting point while keeping your changes safe.

CODEX / 27 Oct 2026 · 17:59

I checked GitHub: the connected repository's default branch is `feature/core-auth`. It has no branch named `main` or `master`.

To switch to that branch while saving your frontend changes:

POWERSHELL

```
git stash push -u -m "Save frontend work"
git switch feature/core-auth
git pull --ff-only
```

Then create your new branch and restore the changes:

POWERSHELL

```
git switch -c your-new-branch-name
git stash pop
```

One important detail: your frontend currently uses backend code from `feature/crud-data`. Starting from the default branch may require bringing those CRUD changes into your new branch too.

Your existing `frontend-integration` branch remains available. Nothing has been committed or pushed.

YOU / 27 Oct 2026 · 18:02

i want to check for conflicts in my frontend after pulling

CODEX · progress update / 27 Oct 2026 · 18:02

I'll check your current branch, whether the pull completed, and whether Git reports any conflicts. I'll also inspect the frontend changes for compatibility with the backend now on your branch.

CODEX · progress update / 27 Oct 2026 · 18:02

Git reports no unresolved conflicts, and I found no conflict markers in the project files. You're on `fe-integration`, based on the pulled default branch at `78e18ed`. That branch now includes the CRUD merge, so the frontend's storage backend is present. I'll run the integration checks next to look for issues Git wouldn't catch.

CODEX · progress update / 27 Oct 2026 · 18:03

All 369 focused checks passed, including authentication, storage, encryption, and ownership protection. The frontend JavaScript syntax checks and Git whitespace check also passed. I haven't changed any source files during this check.

CODEX / 27 Oct 2026 · 18:04

Your frontend has **no unresolved Git conflicts** after pulling.

- `fe-integration` starts from the pulled default branch, which now includes the CRUD backend.
- No conflict markers were found.
- **369 integration and security tests passed**, plus JavaScript syntax checks. The known SQLite cleanup warning was suppressed.
- Your frontend changes are still uncommitted and nothing was pushed.

Restart Flask and check login, upload, comparison, download, and deletion in your browser to confirm the visual behavior too.

YOU / 27 Oct 2026 · 18:04

are there things that need to be added to the gitignore?

CODEX · progress update / 27 Oct 2026 · 18:05

I'll check `.gitignore` against the project's local configuration, storage files, and generated files.

CODEX / 27 Oct 2026 · 18:05

Your `.gitignore` already covers the important files: `.env`, virtual environments, Python caches, databases, and `instance/` storage.

I'd add these for the temporary test folders we've used and SQLite companion files:

GITIGNORE

```
# Temporary test runs
/.pytest-*/

# SQLite companion files
*.db-journal
*.db-wal
*.db-shm
```

Keep `app/static/`, the generated medical images, templates, and `design.md` tracked—they're part of the frontend.

I haven't changed `.gitignore` yet.

YOU / 27 Oct 2026 · 18:17

is it possible to show a preview of the uploaded file in the fe

CODEX / 27 Oct 2026 · 18:18

Yes. We can add a **Preview** button beside each stored file:

- **JPG/PNG:** display the image.
- **PDF:** open a document viewer.
- **MP4:** show a video player.
- **Word/Excel:** offer download; browser previews would require conversion.

For stored files, we'd add a small backend preview route that checks ownership, decrypts the AES copy, and returns the correct file type. The current download route always forces a download.

The preview could open in a modal, load only when requested, and clear when closed. We'd also need to account for the current Content Security Policy when embedding PDFs.

YOU / 27 Oct 2026 · 19:25

what is the evaluation method use here to compare the encryption methods. how are they benchmarked

CODEX · progress update / 27 Oct 2026 · 19:25

I'll check exactly what the encryption code times and what the comparison table calculates, so I can explain the evaluation method accurately.

CODEX / 27 Oct 2026 · 19:26

Currently, the app performs a **single-run comparison for each uploaded file**, rather than a repeated benchmark.

Every upload encrypts the **same original bytes** with all three algorithms:

Algorithm	Configuration
AES	256-bit key, CBC mode, padding

Algorithm	Configuration
DES	8-byte key, CBC mode, padding
RC4	128-bit key, stream cipher, discards the first 1,024 keystream bytes

The comparison measures:

- **Encryption time:** Python's `time.perf_counter()` measures one call to the encryption function. This includes cipher setup, IV generation and padding where applicable. It excludes uploading, key generation, authentication-tag creation, key wrapping, and saving files.
- **Encrypted size:** the stored ciphertext size in bytes, including the **32-byte authentication tag**. Database metadata, IVs, and wrapped keys are excluded.
- **Storage overhead:** `encrypted size − original size`. The percentage is `overhead ÷ original size × 100`.
- **Correctness:** recovered files are checked against the original SHA-256 hash. Tests also verify that downloads from all three copies reproduce the original bytes.

The measurements are saved when the upload happens. Opening the comparison table reads those saved values—it doesn't rerun encryption.

Its current limitations are **one measurement per algorithm, a fixed order of RC4 → AES → DES, no warm-up runs, and no averages or variation statistics**. Decryption time, memory use, and full upload time aren't measured.

For your report, describe this as **"per-upload encryption timing and storage-overhead comparison."** A proper benchmark would add repeated trials across several file sizes, rotate the algorithm order, and report median time, variation, and throughput. Speed results alone do not establish which algorithm is secure.